

Octos Robot Abstraction & Multi-Robot SDK

— Design Proposal

Status: ASPIRATIONAL / DESIGN PROPOSAL. Per

[OCTOS_ROBOTICS_ARCHITECTURE.md](#), design proposals live under `docs/design/` and describe a target, not shipped behavior. Every code block marked **PROPOSED** is not implemented. The "Exists today" section is the only part that describes current `main` behavior, and it is verifiable against the tree.

See also:

- [OCTOS_ROBOTICS_ARCHITECTURE.md](#) — architectural targets (T1–T3), required invariants
- [OCTOS_ROBOTICS_CONTRACTS.md](#) — per-issue contracts R01–R10
- [OCTOS_ROBOTICS_FAMILY.md](#) — program plan, phases, roles
- [OCTOS_ROBOTICS_PR270_RP06_NOTE.md](#) — why a linked-zenoh dora crate was kept off `main`

Purpose

This proposal answers one question the R-family does not yet answer:

What is the abstraction and SDK that lets *multiple, heterogeneous* robots be integrated into Octos the way multiple chat channels already are — each with its own identity, capabilities, safety profile, and lifecycle — without bespoke wiring per robot and without weakening any stable surface?

It is **connective tissue**, not a reshape. The R-family already names the two execution planes a robot needs:

- the **control plane** — LLM-callable, tool-shaped verbs (exists today via the HTTP bridge, see below);
- the **data plane** — high-frequency typed sensor/state streams with backpressure (`R04 RoboticsBridge` , planned).

What is missing is the object that says *"this is robot X: here are its verbs, its streams, its tiers, its namespace, its health"* — and a registry that can hold **N** of them. Today a robot is implicit: a flat tool-map plus (eventually) a per-adapter bridge. You cannot cleanly run two robots, namespace their verbs, gate them independently, or hand a vendor a single contract to integrate one. This

document proposes that object — the `Robot` manifest, trait, and registry — and the SDK around it.

The design is deliberately derivative: it copies three abstractions Octos or its neighbors have **already proven**, and adds only what robots specifically need.

What this borrows, and from where

Source	Proven pattern	What we take
Octos gateway (<code>octos-bus/src/channel.rs</code>)	<code>Channel</code> trait + <code>ChannelManager</code> registry: many transports → one agent core, 3 required methods + defaulted rest, <code>Arc<dyn Channel></code> in a <code>HashMap</code> .	The in-process contract shape and the registry. <code>Robot</code> is to hardware what <code>Channel</code> is to messaging.
openclaw (<code>src/channels/plugins/</code>)	<code>ChannelPlugin</code> = a manifest (<code>openclaw.plugin.json</code>) + fine-grained optional adapters (<code>config / outbound / status / auth / ...</code>), directory discovery, lazy activation by trigger.	The manifest + capability-adapter + discovery model — the right shape when <i>third parties</i> (robot vendors) add integrations, not the platform owner.
dora-rs (<code>apis/</code> , <code>node-hub/</code>)	Minimal node contract (<code>init_from_env</code> → <code>Event</code> stream → <code>send_output</code>); declare I/O by id ; a node-hub library of reusable hardware drivers; dynamic nodes (external process joins a running dataflow); metadata correlation keys (<code>request_id</code> , <code>goal_id</code> , <code>goal_status</code> , <code>seq</code> , <code>fin</code>).	SDK ergonomics ("fill in 3 methods"), the robot-hub philosophy, the streaming + goal/feedback upgrade, and the dynamic-node bridge pattern.

Exists today (verifiable against `main`)

This is the only descriptive section. Everything below it is proposed.

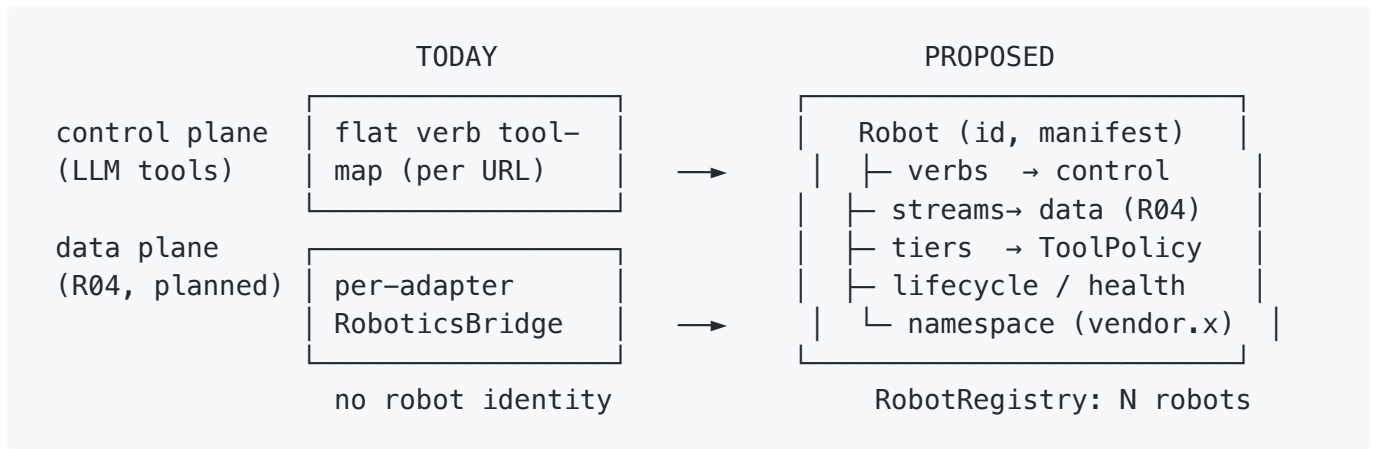
The **control plane** is real and shipped (landed in `#1347`, `crates/octos-agent/src/tools/dora_bridge.rs`):

- A `DoraToolMapping` maps **one robot verb** → **one MCP tool**. The agent's normal `Tool` machinery dispatches it as `POST http://127.0.0.1:<port>/tools/<verb>` with `{"args": ...}` and parses a SPEC-V1 envelope `{ok, code, msg, data}` (`tools/http.rs`).
- An **external bridge process owns the dora runtime** (zenoh/arrow). Octos links none of it and only speaks loopback HTTP — `tools/http.rs` rejects anything that is not a literal `127.0.0.0/8 / ::1` IP (no `localhost`, to kill DNS rebinding) and caps responses at 1 MiB. **This is the resolution of the RP06 concern**: the rejected design linked `dora-arrow / zenoh` into the agent; the shipped design pushes those deps out of process behind loopback HTTP.
- **Safety is a first-class, enforced spine.** `permissions.rs` defines an ordered `SafetyTier` (`Observe < SafeMotion < FullActuation < EmergencyOverride`). `load_bridges()` registers each verb's tier into a global `RobotToolRegistry` (`tools/robot_groups.rs`) so `ToolPolicy`'s `group:robot:<tier>` expansion actually gates calls. Actuation tiers default to `ConcurrencyClass::Exclusive` — overlapping motion commands cannot run in the same parallel batch.
- **Namespacing primitive already present.** `verb_path` keeps the original dotted SPEC-V1 verb (`vendor.agibot.a2.motion.set_action`) on the URL while exposing a provider-legal sanitized name to the LLM.
- **Discovery, two ways**: static `BridgeConfig::from_file(...)`, or a plugin manifest with `tool_discovery: {type:"http", base_url}` → `plugins/http_discovery.rs` pulls `GET /tools` at startup.

What does **not** exist today, and bounds the proposal:

- **No Robot object.** `dora_tool_map.json` is a flat, undifferentiated verb list. Two robots are two URL prefixes with no first-class boundary, no per-robot tier policy, no per-robot health, no namespacing guarantee.
- **No streaming / feedback.** Every verb is a synchronous call with a timeout. The agent cannot see `moving... 40%... done`; it fires blind.
- **No data plane wired.** `R04`'s `RoboticsBridge` (sensor streams, QoS) is contracted but not yet landed.

The gap, precisely



R04 gives a robot a *pipe*. R10 (cell orchestration, Phase RD, deferred) assumes a `Robot` type already exists and coordinates several of them. **This proposal supplies the `Robot` type that sits between them** — the thing R04 streams into and R10 schedules over.

Design principles

1. **A `Robot` is identity + manifest + capabilities, not a transport.** It *composes* the existing control-plane bridge and the planned R04 data plane under one namespaced handle. One physical robot = one `Robot` = one `RobotManifest`. The transports stay where they are.
2. **Reuse the safety spine verbatim.** Verbs register into the existing `RobotToolRegistry` at their declared tier; `ToolPolicy` and `ConcurrencyClass` gate them unchanged. No second safety system.
3. **Manifest-driven, third-party extensible.** A robot vendor ships a manifest + a thin adapter and drops it in a directory — like a skill, like an openclaw extension. Adding a robot is config + a small adapter, **not a core recompile** and **not framework surgery**.
4. **Minimal contract, dora-style.** A robot author fills in a handful of methods, the way a `Channel` author or a dora node author does.
5. **Honor every R-family invariant.** No prompt-carried safety, no unbounded buffers, no stub crates, no Python port, no `unsafe`, no new path that bypasses the sandbox / `ToolPolicy` / `BLOCKED_ENV_VARS`. Streaming does **not** link zenoh/arrow into the agent (see §"Streaming" tradeoff).

Architectural surface (PROPOSED)

`RobotManifest` — the declaration (openclaw lesson)

Promote the flat `dora_tool_map.json` into a per-robot manifest with identity, namespace, capabilities, and lifecycle. Discovered from a directory and activated lazily, like skills.

```
// PROPOSED – robots/agibot-a2/robot.json (not implemented)
{
  "id": "agibot.a2",                      // stable, dotted, vendor-namespaced
  "display_name": "AgiBot A2 (left cell)",
  "control": {                            // control plane – the existing HTTP
bridge
    "bridge_base_url": "http://127.0.0.1:8765",
    "verbs": [
      { "name": "motion.set_action", "safety_tier": "safe_motion",
        "input_schema": { "type": "object", "properties": { /* ... */ } },
        "timeout_secs": 30 },
      { "name": "gripper.rotate",      "safety_tier": "full_actuation",
        "concurrency_class": "exclusive", "timeout_secs": 45 },
      { "name": "camera.capture",      "safety_tier": "observe" }
    ]
  },
  "data": {                               // data plane – delegated to R04
RoboticsBridge
    "adapter": "dora",
    "streams": [
      { "topic": "tcp_pose", "qos": { "drop_oldest": 4 }, "schema":
"std/geom/v1/Pose" },
      { "topic": "ft_sensor", "qos": "best_effort",          "schema":
"std/force/v1/Wrench" }
    ]
  },
  "lifecycle": { "connect_timeout_ms": 5000, "health_topic": "status" },
  "estop_verb": "safety.estop"           // dedicated, tier-bypassing
}
```

The control-plane half is exactly today's `DoraToolMapping` fields, just grouped under a robot. The data-plane half is a *reference* to `R04` topics — this proposal does not re-implement the bridge.

Robot trait — the in-process contract (Channel lesson + dora minimalism)

```
// PROPOSED – crates/octos-agent/src/robot/mod.rs (not implemented)
#[async_trait]
pub trait Robot: Send + Sync {
    /// Stable namespaced id, e.g. "agibot.a2". Used to namespace every verb.
    fn id(&self) -> &RobotId;

    /// Declared capabilities: verbs (control) + streams (data) + tiers.
    fn manifest(&self) -> &RobotManifest;

    async fn connect(&self) -> eyre::Result<()>;
    async fn disconnect(&self) -> eyre::Result<()> { Ok(()) }

    /// Control plane: invoke one verb. Today's HTTP bridge path.
    async fn invoke(&self, verb: &str, args: &serde_json::Value)
        -> eyre::Result<ToolResult>;

    /// Data plane: subscribe to a declared stream. Delegates to the R04
    /// `RoboticsBridge`; the agent never holds an unbounded buffer.
    async fn subscribe(&self, topic: &str)
        -> eyre::Result<BoxStream<'static, RobotEvent>>;

    /// Dedicated emergency stop. Tier-bypassing, deterministic, never an LLM
    /// decision (composes with R03 supervisor when present).
    async fn estop(&self) -> eyre::Result<()>;

    async fn health(&self) -> eyre::Result<RobotHealth>;
}
```

invoke + safety + concurrency reuse **exactly** what dora_bridge.rs , robot_groups.rs , and permissions.rs do today. subscribe / estop / health are the additions dora teaches. The contract is intentionally small — the dora lesson is that a node author implements ~3 things.

RobotRegistry — holding N robots (Channel/ ChannelManager lesson)

```
// PROPOSED - crates/octos-agent/src/robot/registry.rs (not implemented)
pub struct RobotRegistry {
    robots: HashMap<RobotId, Arc<dyn Robot>>,
}

impl RobotRegistry {
    /// At startup, for each robot, register every verb into the EXISTING
    /// RobotToolRegistry at its tier, namespaced "<robot_id>.<verb>" so two
    /// robots never collide. ToolPolicy group:robot:<tier> gates them all.
    pub fn install_into_tool_registry(&self, tools: &mut ToolRegistry) { /* ...
*/ }
}
```

This is structurally identical to `ChannelManager` (`channel.rs:264`): a `HashMap<id, Arc<dyn _>>`, registered at startup, dispatched by id. **Multiple robots = multiple namespaced registry entries**, each with its own bridge URL, tier profile, and health, all gated by the one `ToolPolicy`.

The dora bridge becomes *one* `Robot` impl (dora node-hub lesson)

```
crates/octos-agent/src/robot/
hub/
  dora.rs      // DoraRobot    - wraps today's HTTP bridge + R04 dora adapter
  ros2.rs      // Ros2Robot    - R04 ros2 adapter + ROS2 service verbs
  mavlink.rs   // MavlinkRobot - autopilot
  serial.rs    // SerialRobot  - direct device, SPEC-V1 over a local server
```

A **robot-hub** mirroring dora's `node-hub` and Octos's own channel-adapter directory (`octos-cli/src/commands/gateway/adapters/`). Adding a robot family is one small adapter file, not a new framework.

The SDK (PROPOSED)

Two halves, exactly like "dora node SDK + openclaw plugin scaffold".

Robot-side SDK — what a vendor builds to expose their robot:

- `octos robot new <template>` (à la `dora new`) scaffolds either
 - **(a)** an HTTP server speaking the existing SPEC-V1 `{ok,code,msg,data}` control contract (which `dora_bridge.rs` already consumes) plus an optional loopback stream endpoint; or

- **(b)** a dora dataflow where **Octos joins as a dynamic node** (dora's `path: dynamic / init_from_node_id`) for vendors already on dora.
- A `robot.json` JSON-Schema + `octos robot validate` (mirrors how skill manifests are validated).

Octos-side SDK — what the platform provides:

- the `Robot` trait, `RobotManifest` / `RobotId` / `RobotEvent` / `RobotHealth` types, and `RobotRegistry`;
- reuse of the existing `SafetyTier` / `ToolPolicy` / `ConcurrencyClass` / `RealtimeConfig` machinery;
- the robot-hub adapters above.

Streaming & feedback — the one real capability gain (dora lesson)

Today's bridge is synchronous request/response; it cannot express progress or long-running actuation feedback. dora already standardized the vocabulary: `goal_id`, `goal_status`, `seq`, `fin`. Adopt those on `RobotEvent` so a verb like `motion.set_action` can stream `accepted` → `executing(40%)` → `succeeded`.

Tradeoff (must honor RP06): there are two ways to get streaming.

- **Link dora into the agent** (zenoh/arrow). Richest, but reintroduces exactly the dependency + trust surface the [RP06 note](#) deliberately rejected, and the `R04` invariant that the data plane is **not** an LLM-visible tool. **Rejected for the control plane.**
- **Loopback streaming** (SSE/WebSocket on `127.0.0.1`, same origin checks as `tools/http.rs`) alongside the existing SPEC-V1 verbs. The external bridge keeps owning dora; Octos gets `subscribe()` + `goal_status` feedback without linking zenoh/arrow into the `deny(unsafe_code)` agent. **Recommended.**

For the high-frequency sensor data plane, streaming is `R04`'s job and stays out of the tool registry; `Robot::subscribe` is a thin, namespaced handle over an `R04` topic, never an unbounded buffer in agent memory.

Worked example — two robots coexisting (PROPOSED)

Drop two manifests in `robots/`:


```
robots/agibot-a2/robot.json    id = "agibot.a2"    bridge :8765
robots/so101-left/robot.json  id = "so101.left"   bridge :8766
```

At startup the registry installs namespaced, tier-gated tools:

```
agibot_a2__camera_capture    observe    (Safe concurrency)
agibot_a2__motion_set_action safe_motion (Exclusive)
so101_left__gripper_rotate   full_actuation (Exclusive)
...
```

A profile policy `allow: ["group:robot:safe_motion"]` grants every robot's `observe` + `safe_motion` verbs by subset semantics, and denies actuation — **per the existing RobotToolRegistry, no new code**. A pipeline mission (`R02 / R05`) can now reference verbs across both robots; a future cell (`R10`) can schedule them. Two robots, one policy, one registry, zero collisions.

What is NOT in this design

Following the architecture doc's "What Is Not Architecture" discipline:

- **Not a replacement for R04**. `Robot::subscribe` delegates to the `R04 RoboticsBridge`. This proposal does not implement a transport, QoS, or schema validation — those are `R04`.
- **Not multi-robot *coordination***. Resources, priorities, deadlock avoidance are `R10` (Phase RD, deferred). This is the `Robot type R10` builds on, not the scheduler.
- **Not a safety system**. Tiers are declared here and enforced by the existing `ToolPolicy` (and, when present, the `R03 supervisor`). A manifest field is not enforcement.
- **Not a new execution loop, session model, or DSL**. The agent loop, tool registry, sessions, and pipeline engine are stable surfaces and are reused.
- **No stub crate**. The robot module lands only when it forwards real verbs to the real, already-shipped HTTP bridge.

Required invariants (compose with the stable surfaces)

1. A `Robot` introduces no execution path that bypasses the sandbox, `ToolPolicy`, `BLOCKED_ENV_VARS`, `O_NOFOLLOW`, or `SafePolicy`.
2. Every robot verb is registered into the existing `RobotToolRegistry` at its declared tier; tier evaluation remains the only gate.

3. Verb names are namespaced by `RobotId` ; two robots cannot shadow each other's tools. Verified by a registration-collision test.
4. The agent links neither `zenoh` nor `dora-arrow` ; control + streaming reach the agent only over loopback HTTP/SSE with the `tools/http.rs` origin checks. Verified by a dependency-graph assertion.
5. `Robot::subscribe` holds no unbounded buffer; backpressure is `R04` 's bounded-channel contract.
6. No `unsafe` . The robot config section is `RestartRequired` in `diff_profiles` , like the other robot sections.
7. The non-robot agent path is byte-for-byte unchanged when no robot is configured (feature off by default).

Proposed contracts (for Architect ratification)

The R-family is governed: contracts are immutable and the Architect owns ratification ([FAMILY.md](#) §Roles). This proposal does **not** author binding contracts. It *sketches* two candidate additions spanning the `R04` → `R10` gap, in the contract template's shape, for the Architect to accept, renumber, or reject.

Candidate R11 — Robot Registry & Manifest (Tier 2, Phase RB/RC; blocked by `R04`). Adds `crates/octos-agent/src/robot/` (`Robot` trait, `RobotManifest` , `RobotRegistry` , namespaced tool install), the `ProfileConfig.robot.fleet` typed section, and directory discovery. Verbs forward to the **already-shipped** HTTP bridge (no stub). Acceptance: namespaced registration, collision rejection, tier subset enforcement across two robots, dependency-graph assertion (no zenoh/arrow in agent).

Candidate R12 — Robot SDK & robot-hub (Tier 2/3; blocked by R11). Adds `octos robot new / octos robot validate` , the `robot.json` schema, and the `hub/` adapters (`DoraRobot` first; others `Unimplemented` errors, not stubs, per `R09` precedent). Adds the loopback streaming endpoint + `goal_status` feedback vocabulary. Acceptance: scaffold produces a robot the registry loads; streaming verb reports progress; unimplemented adapters fail at load, not at call.

Both must pass the standing **Safety Gate** review and the program-level required-invariant set.

Open questions for the Architect

1. **Renumbering.** The family doc fixes identifiers at `R01–R10` and forbids opening contracts outside that graph. Do these land as `R11 / R12` (extending the graph), as `RP07 / RP08` in the delivery family, or folded into the scope of `R04 / R10` ?

2. **Streaming home.** Confirm loopback SSE over linked-zenoh for the control plane (this doc recommends loopback to preserve the RP06 decision).
3. **Manifest vs. profile config.** Robots-as-directory (skill-like discovery) vs. robots-as-`ProfileConfig.robot.fleet[]` (typed config). This doc leans directory discovery for third-party extensibility; the Architect owns the call.

Persistence

This is a design proposal under `docs/design/`. It becomes authoritative only when the Architect ratifies a contract against it and the implementing slice lands green on canary. Until then it describes a target, and the only section that describes current behavior is "Exists today".